

Lab for Cryptography: Symmetric & Asymmetric Encryption¹

In this practical you will learn how **encryption** works, and how *symmetric* and *asymmetric* (*public key*) encryption operate so that a body of sensitive information may be transmitted securely across a network.

The traditional use of cryptography was to make messages unreadable to the enemy during wartime. However the introduction of the computing age changed this perspective dramatically. Through the use of computers, a whole new use for information hiding was evolved. Around the early 1970s the private sector began to feel the need for cryptographic methods to protect their data. This could include 'sensitive information' (corporate secrets), password files or personal records.

Computer encryption is based on the science of *cryptography*, which has been used throughout history. Most forms of cryptography these days rely on computers, simply because a human-based code is too easy for a computer to crack. An encryption algorithm, or *cipher*, is used to encrypt normal text, or *plaintext*. This encrypted text is then known as *ciphertext*. Trying to regenerate the original plaintext from the ciphertext is known as *decryption*. Most computer encryption systems belong in one of two categories:

- *Symmetric* encryption (often referred to as *secret-key*, *private* or *single-key* encryption)
- *Asymmetric* encryption (also known as *public-key* or *two-key* encryption)

The encryption key and the decryption key may or may not be the same. When they are the cryptosystem is called a "symmetric key" system; when they are not it is called an "asymmetric key" system. The most widely known instance of a symmetric cryptosystem is DES (Data Encryption Standard). One of the most widely known implementations of an asymmetric key cryptosystem is RSA.

Note: Practical Report Required!

For this lab, you are expected to write up a concise report on what you did. The main thing is to convey your understanding of each of the steps taken. Any questions asked during the procedure text should be answered and you should provide a summary at the end.

Length: Max 5 pages
Delivery: Presentation-10 minutes
Deadline: 15th July, 2026

1 Secret Key (Symmetric) Encryption

1.1 Background to DES

IBM initially developed the Data Encryption Algorithm in the 1960s. They used concepts that had been described by Claude Shannon in the 1940s and called their technique *Lucifer*. Lucifer was refined, renamed the DEA (Data Encryption Algorithm) and adopted as the standard in 1976.

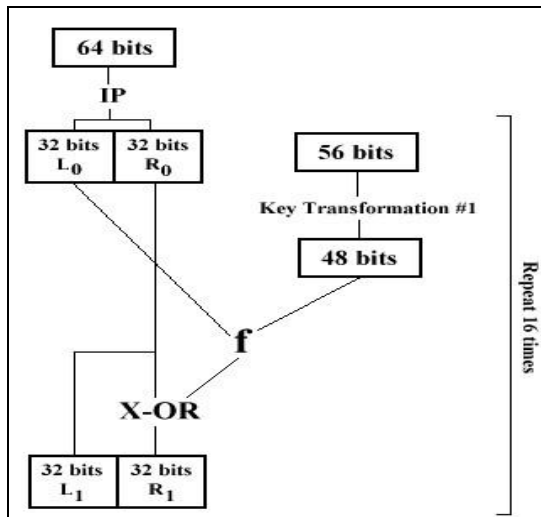
1.2 Implementing DES

The DEA performs a transformation on a block of 64 bits using a 56-bit key. i.e. it takes 64 bits of the plaintext (data to be encrypted) and changes it into a different array of 64 bits (the ciphertext), using a key (known only by the person 'sending' the message and the person 'receiving' it). It does this in several steps, using several kinds of transformations. Crucial to the DEA is the concept of a permutation. This just means that the bits are put in a different order, i.e. jumbled up.

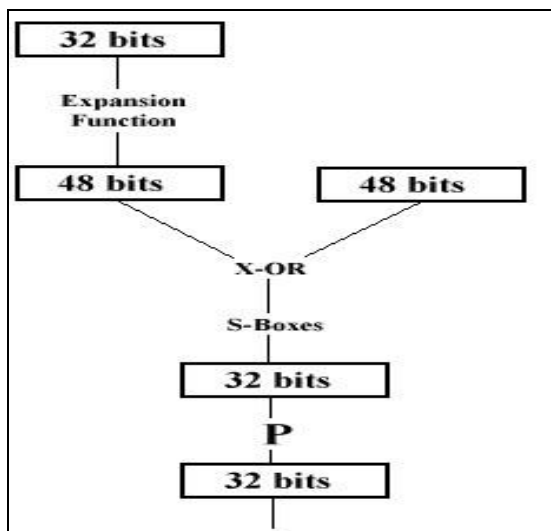
Firstly the Initial Permutation (the IP takes the 64 bits, and changes their order around according to a fixed permutation, so the 58th bit becomes the first bit, the 50th bit becomes the 2nd bit, etc) is applied to the 64-bit plaintext. The result is then divided into two 32-bit halves, named L0 and R0. Then, the following happens 16 times:

Information Security – Lab Project 2

For Iteration Number i (see diagram left):



- Key transformation number i (a permutation, but dropping 8 bits off - defined in the specification) is applied to the key to produce 48 bits.
- Let A be L_i and J be the transformed key. Apply the function $f(A, J)$ (explained below) to produce a 32 bit output.
- Exclusive Or R_i and $f(A, J)$, and call this R_{i+1} .
- Make $L_{i+1} = R_i$



Next (see diagram left), 16 iterations of a function f are applied. f takes 32 bits of the plaintext (A) and 48 bits of the key (J). An expansion function is applied to A , which swaps some of the bits around, and adds an extra 16 of them, which expands it out to 48 bits. The expanded A and J are then combined, using Exclusive Or. This 48 bit block is then put through some S boxes (explained soon) to produce an output of 32 bits. Finally another permutation called P is applied

DES is a fairly weak encryption technique that uses 56-bit keys. It uses the same key to encrypt as to decrypt.

There is only the private key, which both sender and receiver must know. A personal computer could crack DES-encoded message within a year; a group of them could do it proportionately faster. This is one reason why DES is considered less secure than other encryption methods, and why it has evolved into the Advanced Encryption Standard (AES). The advantage DES has over public key encryption is that it is much faster to compute.

1.3 Lab Procedure:

First of all:

1. Organise yourselves into groups of two or three. You will exchange files and keys with the other person(s).
2. Change the settings on Windows to make file extensions viewable.
Launch Windows Explorer, *Tools* menu, select *Folder Options* and then the *View* tab. Then uncheck *Hide extensions for known file types*.
This is not essential, but makes it easier to see what's going on. *Encrypting:*
3. Create a new folder and save the [DES.zip](#) file to it. Extract all files to the folder.
4. Create a new text file with some plaintext as its body e.g "My name is Mary". Save this file in the folder you created.
5. When saving this file select All Types as the "Save as file type" and give your file name the extension ".open" e.g. *jbloggsfile.open*.

Information Security – Lab Project 2

6. At the command prompt simply type **java des** to run the GUI (*N.B. you must first navigate into the folder you extracted the DES.zip file to*).
7. When the GUI opens, generate a random key. Take a note of this key so that it can be used to decrypt the ciphertext
8. Browse and select the *jbloggsfile.open* file that is to be encrypted, and press *Encrypt*
9. An *jbloggsfile.close* file will automatically be generated in the default folder you are working from. View its contents to see what the ciphertext looks like! You may need to terminate the program with ctrl-c to be able to view the contents of the file.
Sharing key & encrypted file:
10. Give your encrypted file (*jbloggsfile.close*) to the other member(s) of your group, either by email or on a disk
11. Write down your (hexadecimal) key and share it with them.

Decrypting:

12. Receive a key and file from each other member(s) of your group. This should have a *.close* extension
13. Try to open their file with Notepad to see if it makes any sense (it shouldn't!)
14. Enter their key
15. Browse and select the their file, and press Decrypt (this is so that the ciphertext generated from the Encryption part above is put through the same algorithm)
16. A new file with a *.open* extension is created in the folder which should have the original plaintext he/she first entered in as its content body in step 4 above *Exercise Question:*
17. Create a new file, this time with a lot of text (say 1MB in total). Call it *bigfile.open*. Encrypt it to give *bigfile.close*. Use *WinZip* to compress both of them. Which resulting *zip* file is bigger? **Why is this the case?**

2 Public Key (Asymmetric) Encryption

2.1 Background to RSA

The acronym stands for Rivest, Shamir, and Adleman, the inventors of the technique. RSA is a very powerful encryption algorithm that is based on the public key encryption method. Public key encryption means that you have two pairs of keys, public and private. You give the public key to everyone and keep the private key to yourself. Then, people can encrypt data using the public key and send it to you. Only you can decrypt the message as only you have the private key; the message cannot be decrypted using the public key.

2.2 Implementing RSA

Two really big prime numbers are needed in order to implement RSA. Prime numbers are numbers that do not divide into anything else other themselves or 1 e.g. 7, 13, 29, 101. Next, we find the product of the two big prime numbers, so let $N = \text{First prime number} * \text{Second Prime number}$. This is one part of the public key. To find the second number we must understand what *relatively* prime numbers are. Relatively prime numbers are numbers that share no factors at all e.g. 7 and 13 are relatively prime, 15 and 5 are not. Now we have to check whether N and $p*q$ are relatively prime i.e. $p = \text{the first prime number} - 1$ and $q = \text{the second prime number} - 1$. To do this, we have to find a number E in which $\text{gcd}(E, p-1*q-1) = 1$.

$\Rightarrow N$ and E together are the public key pair.

Next, we have to find the private key pair. The private key is such: $D = E^{-1} \bmod ((p-1)(q-1))$

¹ gcd = greatest common divisor function $\square \text{gcd}(E, \text{First prime number} - 1 * \text{Second prime number} - 1) = 1$, means the greatest common number which when divided in to all three numbers within the brackets will equal one, which ultimately means E and $(p-1) * (q-1)$ have no common factors except 1

Information Security – Lab Project 2

Mod (modulus) is the remainder after division. So, D is the remainder left after E^{-1} is divided by the product of the first prime number and the second prime number.

$\Rightarrow D$ and E are our private key pair This means that:

$\Rightarrow (N$ and $E)$ is the public key pair and $(D$ and $E)$ is our private key pair.

So, to encrypt a letter A , having S as the ciphertext, it would be done like this – S

$$= A^E \bmod N$$

To decrypt the alphabet and obtain the original text –

$$A = S^D \bmod N$$

2.3 Lab Procedure

1. Extract all files to a new folder from the [RSA.zip](#) file
2. A quick way to see how this works is to open the QuickStartRSA.html file that also has helpful guides to the code that is being implemented by the program. However, running the program from the command line is more useful to see what is happening with the calculations!
3. At the command prompt go to the required directory
4. Run the program by using **java RSA**. You also need to include here, at the end of the command, some value that represents the size in bits of each generated prime number used in the calculations e.g. 8, 16, 32, 64...512 (**java RSA 8**)
5. The prime numbers p and q are then generated (and displayed) as well as the public key pair (N and E) and private key pair (D and E)
6. You are then prompted for some plaintext. Enter something!
7. The ciphertext is displayed.
8. Experiment with changing the size of the prime number to see the variation in the ciphertext and the difference in the degree of difficulty that would occur when trying to decipher the two

3 Performance Testing

As you will have seen, encryption and decryption take time, especially on slow computers. In this section, you will benchmark the performance of the applications chosen.

3.1 Lab Procedure

DES Performance

1. Create 4 or 5 test files of varying sizes, from a few bytes to about 500KB.
2. For each file, record how much time it takes to (a) encrypt and (b) decrypt using DES
3. Tabulate and graph your results in your report
4. On average, how many bits per second can this encryptor process on your machine? Comment.

RSA Performance

5. Run 4 or 5 times, varying the size of the prime numbers used (number of bits) and record how much time it takes
6. Tabulate and graph your results in your report
7. Comment

4 Lab Question 3: Biometric-Based Access to Encrypted Data or Any Message

Biometric Integration Task:

In this task, you are required to simulate a basic biometric authentication mechanism (e.g., using a fingerprint image or face recognition) to protect access to your encrypted file or message from Question 1 or 2.

Instructions:

- Simulate user authentication using a biometric input (e.g., a fingerprint image, or your face via webcam).
- Only allow access to the decryption key or decrypted message if biometric authentication is successful.
- You may use any simple biometric simulation libraries (e.g., face_recognition, OpenCV) to complete this task.

Questions to Answer:

- a. What type of biometric input did you simulate?
- b. How did you use it to control access to your cryptographic key or message?
- c. What are the potential security risks in using biometric authentication in this way?